



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

F# PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Programming Languages

TOTAL: 10 QUESTIONS

Q1 What are the primary design goals of F#?

Threat Model

Q6 How does F# implement object-oriented or functional programming?

Observability

Q2 How does F# handle type checking: static or dynamic?

Architecture

Q7 How does F# handle errors?

Lifecycle

Q3 How does F# manage memory?

Configuration

Q8 What testing tools are commonly used in F#?

Compliance

Q4 What is F#'s concurrency model?

Hardening

Q9 What makes F# well-suited for its target domain?

Testing

Q5 How are packages and dependencies managed in F#?

SSO / IdP

Q10 What are common performance pitfalls in F#?

Tuning



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 F# was designed with specific priorities around

Mitigation

F# was designed with specific priorities around performance, safety, or developer productivity depending on its domain. These goals shape its syntax, type system, and standard library choices.

A6 F# supports classes and inheritance for OOP,

Observability

F# supports classes and inheritance for OOP, or first-class functions and immutability for FP, or both. Many modern F# programs blend both paradigms depending on the problem.

A2 F# uses its type system to catch

Primitives

F# uses its type system to catch errors at compile time (static) or at runtime (dynamic). This affects refactoring safety, tooling support, and the kinds of bugs that reach production.

A7 F# surfaces errors via exceptions, Result/Either types,

Automation

F# surfaces errors via exceptions, Result/Either types, or error return values. The choice impacts whether errors are checked at compile time and how calling code is structured.

A3 F# uses one of three approaches: manual

Hardening

F# uses one of three approaches: manual allocation/deallocation, a garbage collector that periodically reclaims unreachable objects, or automatic reference counting. Each has different latency and throughput tradeoffs.

A8 F# has a standard or widely adopted

Governance

F# has a standard or widely adopted test runner and assertion library. Tests are typically organized into unit, integration, and end-to-end layers with coverage reporting built in or via plugins.

A4 F# supports concurrency through threads, coroutines,

Gotchas

F# supports concurrency through threads, coroutines, async/await, or an actor model. The choice determines how I/O-bound and CPU-bound workloads are scaled and how shared state is safely accessed.

A9 F# excels in its domain due to

Assessment

F# excels in its domain due to a combination of performance characteristics, ecosystem maturity, language ergonomics, and community support that makes common tasks simple.

A5 F# uses a package manager and a

Federation

F# uses a package manager and a manifest file to declare dependencies and version constraints. A lock file pins exact versions to ensure reproducible builds across environments.

A10 Typical performance issues in F# include excessive

Optimization

Typical performance issues in F# include excessive allocations, blocking the main thread or event loop, $O(n^2)$ data structures, and missing caching. Profiling and benchmarking tools are available to diagnose them.